AI Usage and Disclosure

Student: Nguyet Tram Dinh

Date of AI use: September 27, 2026

AI tool: ChatGPT (GPT- 5.6 Luna)

1. General Disclosure

I used ChatGPT to help generate the first version of my C++ program. I provided the assignment requirements and used AI to help create the parsing logic, review the requirements, think of test cases, and explain parts of the implementation. I reviewed the generated code and made sure I understood how it worked before using it.

2. Code Attribution

ChatGPT generated the first version of the C++ solution, including the candidate-scanning logic, manual digit accumulation, IPv4 and port validation, 32-bit address conversion, and input loop. I reviewed the code against the assignment requirements. The final submitted source should reflect any changes I make after testing.

3. Prompts and Their Effects

#	Prompt Used	Effect Content
1	Generate a C++ program that extracts one valid IPv4 address and optional port from a line of text. Do not use stoi, atoi, sscanf, regex, inet_pton, or other prohibited conversion/parsing functions	Generated the first program structure and parsing functions
2	Check the code against the requirements: four octets, each 0–255, no leading zeros, and an optional port from 0–65535	Focused the review on range checks, octet count, leading zeros, and port validation
3	Check how garbage characters work. what should happen with 192a168.1.1.1?	Clarified that non-token characters separate

		candidates, so 168.1.1.1 can be found after the letter a
4	Check for partial-match problems such as 192.168.1.1. and 192.168.1.1:99999	Confirmed that an invalid full candidate must be rejected instead of extracting a valid-looking part from it
5	Generate test cases including valid inputs, boundary values, invalid octets, leading zeros, invalid ports, extra punctuation, garbage text, and incomplete addresses	Expanded testing beyond the sample run and covered malformed and boundary inputs
6	Explain the manual digit conversion value = value * 10 + (token[pos] - '0')	Helped verify how the program builds numeric values without using string-to-number conversion functions
7	Explain how the four octets are converted into one 32-bit unsigned value using bit shifting	Helped verify the address conversion and understand the 24-, 16-, 8-, and 0-bit positions
8	Review the program for cases where it might accept an invalid address. Suggest tests instead of rewriting the whole program	Used AI as a reviewer and helped identify areas that needed extra testing

4. Testing and Verification

I checked the generated solution against the assignment requirements instead of assuming it was correct because it compiled. I also checked that it did not use prohibited parsing or conversion functions. I used valid, invalid, and boundary inputs such as:

- 192.168.1.1 - valid address with no port
- 10.0.0.255:8080 - valid address with a port
- 0.0.0.0 - minimum octet values
- 255.255.255.255:65535 - maximum octet and port values
- 256.1.1.1 - invalid octet

- 192.168.01.1 - invalid leading zero
- 1.2.3.4:65536 - invalid port
- 192.168.1.1. - extra period makes the candidate invalid
- 1.2.3 - incomplete address
- 192a168.1.1.1 - garbage character separates candidates

5. Modifications and Review

I reviewed the candidate parsing, manual digit conversion, range checks, leading-zero checks, optional port handling, and 32-bit conversion. I did not make a major structural change to the initial implementation because my review and testing did not identify a required structural change. If I make additional changes after testing, I will document those changes and the reason for them in the final version of this disclosure.

6. Verification Statement

I understand the submitted code and can explain how its functions work. I reviewed the AI-generated implementation, checked it against the assignment requirements, and tested it with valid, invalid, and boundary inputs. At the time of this document, I am not aware of any unresolved bugs or unexpected behavior.